

Здесь собраны все технические требования из задания и указаны конкретные файлы, где они реализованы.

1. Архитектурное разделение (SOLID)

- **Разделение на проекты:** Логика полностью отделена от интерфейса.
 - `Model.csproj` (DLL) — Бизнес-логика.
 - `DoodleJump.csproj` (WPF) — Визуализация.
- **Папки Core и Data:** В проекте `Model` файлы разложены по папкам согласно ТЗ Продвинутой части.

2. Абстрактные классы (3 реализовано, нужно было 2)

- `GameObject.cs`: Базовый шаблон для всех предметов в мире (координаты, размеры).
- `Platform.cs`: Базовая логика для всех площадок.
- `BaseSerializer.cs`: Обобщенная база для систем сохранения (JSON и XML).

3. Интерфейсы (2 реализовано, нужно было 2)

- `IUpdatable.cs`: Применяется к объектам, которые меняются во времени (Игрок). Используется в движке для массового обновления.
- `IPlatform.cs`: Описывает свойства площадок (например, активность). Позволяет интерфейсу работать с платформами, не зная их конкретного вида.

4. Полиморфизм и переопределение (Override)

- **Файлы:** `NormalPlatform.cs`, `SpringPlatform.cs`, `TrapPlatform.cs`.
- **Применение:** Метод `OnCollision` переопределен везде. На обычном камне — простой прыжок, на грибе — высокий, на треснувшем камне — прыжок + поломка.
- **Метод `Update`:** Переопределен в `Player.cs` для расчета гравитации.

5. Перегрузка методов и операторов

- **Перегрузка оператора:** Файл `Vector2D.cs`. Перегружены `+` и `-` для работы с координатами.
- **Перегрузка метода:** Файл `Player.cs`. Два метода прыжка: `Jump()` (стандарт) и `Jump(double force)` (с указанием силы).

6. Делегаты и События

- **Файл:** `GameEngine.cs`.
- **Применение:** Используется встроенный делегат `Action` для события `OnGameOver`. Когда игрок падает, Модель через это событие «кричит» Интерфейсу, что пора показывать экран смерти.

7. Обобщенные типы (Generics)

- **Файлы:** `BaseSerializer.cs`, `JsonGameSerializer.cs`, `XmlGameSerializer.cs`.
- **Применение:** Классы умеют сохранять любой тип данных `<T>`. Мы используем это и для игрового процесса (`GameState`), и для настроек (`AppConfig`).

8. Приведение к базовому классу и интерфейсу (5+ раз)

- **Файл:** `MainWindow.xaml.cs`, метод `Draw()`.
- **Где именно:**
 1. `(IUpdateable)_engine.Player` (к интерфейсу).
 2. `(GameObject)_engine.Player` (к базовому классу).
 3. Цикл `foreach (Platform p in ...)` (к базовому классу).
 4. `(IPlatform)p` внутри цикла (к интерфейсу).
 5. При передаче `p` в метод `UpdateOrCreateVisual(GameObject obj...)`.

9. Частичные (Partial) классы

- **Файлы:** `GameEngine.cs` и `GameEngine.Logic.cs`.
- **Применение:** Логика движения отделена от вспомогательных методов и сообщений.

10. Сериализация (JSON и XML)

- **Файлы:** `JsonGameSerializer.cs` и `XmlGameSerializer.cs`.
- **Применение:** Сохранение сессии при выходе в меню.
- **Фишка ТЗ:** В `SettingsWindow.xaml.cs` реализована автоматическая конвертация (копирование) данных из одного формата в другой при смене настроек.

11. Визуальные инструменты (8 видов)

В `MainWindow.xaml` и `StartWindow.xaml` использованы:

1. `Canvas` (игровое поле).
2. `DataGrid` (таблица рекордов).
3. `Slider` (выбор сложности).
4. `ComboBox` (выбор формата).
5. `Button` (кнопки с триггерами).
6. `Image` (персонаж и фоны).
7. `TextBlock` (текст и очки).
8. `Viewbox` (авто-масштабирование окна).